Project 1: World and Launch Documentation

Bryan Tran, Donovan Semenuk

1 World File (`world.sdf`)

The world file defines the Gazebo simulation environment: physics, plugins, and geometry. What the world looks like can be seen in Figure 1 below:

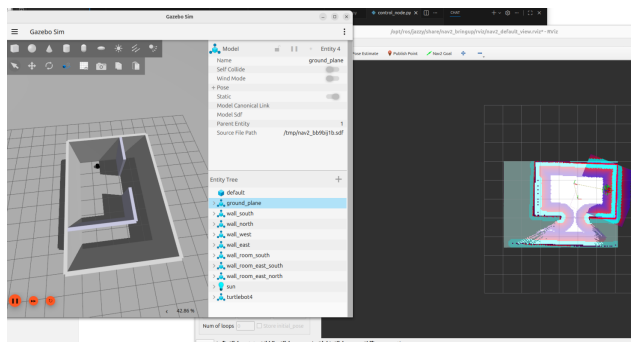

Figure 1: Gazebo environment and RVis map of the world

Physics

This code was provided in "Project_1_Guide.pdf" which was provided via Canvas. This code runs ODE physics at 333 steps/second, real-time speed:

```
<physics name="1ms" type="ode">
  <max_step_size>0.003</max_step_size>
  <real_time_factor>1.0</real_time_factor>
</physics>
```

Plugins

Plugins enable core simulation features, which was again, provided via "Project_1_Guide.pdf":

```
<plugin filename="gz-sim-physics-system" name="gz::sim::systems::Physics"/>
<plugin filename="gz-sim-user-commands-system" name="gz::sim::systems::UserCommands"/>
<plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaster"/>
<plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>
<plugin filename="gz-sim-imu-system" name="gz::sim::systems::Imu"/>
```

The Physics plugin handles rigid-body collision and motion. UserCommands allows external tools to modify the scene. SceneBroadcaster streams the 3D scene to RViz and the Gazebo GUI. Sensors enables simulated hardware like the lidar, using the ogre2 render engine. IMU enables the robot's inertial measurement unit.

Room Layout

Four outer walls form a ~15x20 ft rectangle. Three interior segments create a partition with a doorway:

```
<!-- Outer boundary -->
<model name="wall_south">       <!-- bottom edge, 4.57m wide -->
<model name="wall_north">       <!-- top edge,    4.57m wide -->
<model name="wall_west">        <!-- left edge,   6.1m tall  -->
<model name="wall_east">        <!-- right edge,  6.1m tall  -->

<!-- Interior partition (light blue) with doorway gap -->
<model name="wall_room_south">       <!-- horizontal segment -->
<model name="wall_room_east_south">  <!-- lower vertical segment -->
<model name="wall_room_east_north">  <!-- upper vertical segment -->
```

The room layout can be seen in Figure 1 at the beginning of the document.

2 Launch File (`launch.py`)

The launch file starts Gazebo, Nav2, and SLAM Toolbox in one command by delegating to Nav2's built-in TurtleBot4 simulation launch script.

Launch Arguments

```
world    -- path to .sdf file   (default: project1/worlds/world.sdf)
slam     -- enable SLAM mapping  (default: True)
headless -- hide Gazebo GUI      (default: False)
```

What It Starts

Passes arguments to `tb4_simulation_launch.py` from `nav2_bringup`, which starts:

```
Gazebo simulator      -- loads world.sdf
TurtleBot4 model      -- spawned at (x=2.0, y=4.0) - This is rougly the middle of the inner room
Nav2 navigation stack -- path planning and obstacle avoidance
SLAM Toolbox          -- builds map in real time from /scan
```

3 Run Instructions

All commands are run from the project root folder. Each step requires its own terminal.

Terminal 1 : Build and Launch

Create a folder called "project/" or anything you really want. The documentation will use project/ to refer to the root folder where the code will be.

In project/ clone the repository:

```
    git clone https://github.com/bryanltran/robot-project-1.git
```

Now, we will cd into the root folder and build the project:

```
cd project
colcon build --symlink-install
source install/setup.bash
ros2 launch project1 launch.py
```

Wait for Gazebo to finish loading before continuing.

Terminal 2 : Autonomous Controller

The following instructions will launch the project controller logic. In a new terminal:

```
cd project
source install/setup.bash
ros2 run project1_control project1_controller
```

Terminal 3 : Keyboard Control (Optional)

This is for keyboard control. In a new terminal:

```
cd project
source install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
    --ros-args -r /cmd_vel:=/cmd_vel_key
```

The topic remap (`-r /cmd_vel:=/cmd_vel_key`) is required so keyboard commands go through the controller's priority system rather than bypassing it. Releasing all keys returns control to the autonomous controller after a short timeout.